

Capability-Oriented Software Engineering (COSE): An Incremental Methodology for Building Reusable Capability Libraries

Enrique Solis Carvajal
Change Tensors Solutions
Systems Architecture & Engineering
Email: esolis@tcgplanet.cl

Abstract—Despite decades of progress in software engineering paradigms, software development projects frequently suffer from architectural re-invention, encapsulating functional knowledge within individual project silos. This paper presents *Capability-Oriented Software Engineering* (COSE), a formal incremental methodology designed to systematically convert software engineering effort into a cumulative, permanent library of reusable, decoupled capabilities. COSE establishes a mathematical framework for problem decomposition into single-responsibility micro-problems, resolved via deterministic or cognitive capabilities assembled into execution pipelines. Under COSE, large language models (LLMs) are treated strictly as interchangeable, bounded cognitive components within deterministic control structures, preserving model independence and auditability via persistent file-based artifacts. We evaluate COSE across a diverse spectrum of real-world production systems: *Mammon* (an algorithmic trading platform validated across a 36-configuration stress matrix and live-paper trading on ETHUSD), *Knowledge Factory* (a hybrid cognitive distillation pipeline using structured JSON tensors), *Website Intelligence Factory* (WIF, web reverse-engineering and automated crawler generation), an multi-model AI Middleware orchestrator, and commercial business applications (*Dol-Goldur*, *CatalogVision*, and *Andigraf*). The empirical evidence demonstrates complete infrastructure decoupling, robust operational stability under variable execution parameters, and a progressive reduction in the marginal cost of building complex software systems through composition rather than reconstruction.

Index Terms—Software Engineering Methodology, Reusable Capabilities, Multi-Agent Systems, Pipeline Composition, Deterministic-Cognitive Decoupling, Large Language Models.

I. INTRODUCTION

Software engineering has experienced continuous evolution over the past decades. Paradigms such as structured programming, object-oriented design, service-oriented architecture (SOA), microservices, and multi-agent systems have progressively aimed to manage the complexity of building sophisticated computing solutions. However, a recurring inefficiency persists: each new software project frequently restarts substantial portions of functional design, even when similar problems have already been solved in previous contexts.

In traditional development workflows, knowledge acquired during implementation remains encapsulated within project-specific codebases, application-bound components, and tacit

developer experience. Consequently, organizations lack a systematic mechanism to lower the marginal cost of future software construction.

The methodology presented in this paper did not originate from an abstract theoretical exercise in multi-agent systems or software architecture. It emerged inductively from industrial practice. The author, trained as a business administrator without formal education in computer science or software engineering, began experimenting with large language models (LLMs) and software construction approximately ten months prior to this work. An initial system, informally named *Los Cabros*, was designed around an organizational analogy: interchangeable “characters” (inspired by narrative archetypes), each possessing distinct skills, to be composed dynamically according to the requirements of a given task. This early multi-agent intuition reflected a business-oriented understanding of specialization and collaboration—an approach later documented in the ecosystem context that serves as the workspace within which the COSE projects operate. In practice, however, the character-based approach proved over-qualified for many of the concrete problems being addressed; individual agents frequently concentrated capabilities that could be executed more simply.

A decisive shift occurred during the development of *Dol-Goldur*, an automation system for catalog enrichment and website publication for a physical music retail operation. Instead of assigning broad skill sets to agents, the workflow was decomposed into fine-grained, single-responsibility micro-capabilities arranged in an explicit pipeline. Subsequent systems—*CatalogVision* (image-to-inventory processing) and *Andigraf* (competitive price intelligence and reporting)—confirmed the same pattern: micro-capabilities proved easier to audit, debug, and, crucially, reuse across projects, in contrast to the more complex and less transferable character constructs. The repeated observation that functional units developed for one domain could be composed into solutions for another led to the recognition that a systematic methodology was emerging.

This methodology was then formalized as **Capability-Oriented Software Engineering (COSE)** and subsequently applied to larger-scale systems, including *Website Intelligence Factory* (web reverse-engineering and crawler generation),

This research was conducted within the COSE Multi-Agent and Systems Architecture Infrastructure.

Knowledge Factory (hybrid cognitive distillation), *Mammon* (algorithmic trading infrastructure), and a multi-model AI *Middleware* orchestrator. COSE therefore rests on four core tenets that crystallized from this trajectory:

- 1) **Systematic Problem Decomposition:** Complex domain problems are recursively decomposed into discrete, single-responsibility micro-problems.
- 2) **Atomic Capabilities:** Micro-problems are resolved by minimal, autonomous functional units termed *capabilities*, categorized strictly as either *deterministic* (invariant code, rules, transformations) or *cognitive* (LLM-driven reasoning, judgment, or language synthesis).
- 3) **Cumulative Capability Library:** Capabilities do not belong to specific projects; they are permanently registered into a central, evolving *Capability Library*.
- 4) **Dynamic Pipeline Composition:** Complex software applications are assembled dynamically by chaining existing capabilities into execution pipelines governed by persistent disk-based artifacts.

In COSE, the fundamental unit of organizational value shifts from the transient application deliverable to the permanent, cumulative *Capability Library*. The methodology maintains strict technology and model independence: LLMs serve as replaceable cognitive engines within deterministic control structures, preventing architectural lock-in and ensuring deterministic execution bounds.

The remainder of this paper is organized as follows: Section II reviews related work. Section III formalizes the theoretical framework, mathematical notation, and glossary. Section IV details the COSE architectural principles and execution life-cycle. Section V presents empirical evaluations across seven real-world systems spanning quantitative finance, knowledge distillation, web reverse engineering, cognitive middleware, and commercial vision/catalog processing. Section VI offers a comparative analysis against existing paradigms, and Section VII concludes with key insights and future research directions.

II. RELATED WORK

The paradigm of Capability-Oriented Software Engineering (COSE) intersects three major domains of software engineering and artificial intelligence: multi-agent systems, tool-augmented language models, and architectural patterns for reusability.

A. Multi-Agent Systems and Agentic Frameworks

Distributed Artificial Intelligence and Multi-Agent Systems (MAS) established the foundations for autonomous agent cooperation and task allocation [1]. With the advent of Large Language Models (LLMs), agentic frameworks such as ReAct [2] and AutoGen [3] demonstrated effective reasoning, planning, and multi-agent communication. However, these frameworks primarily focus on dynamic task execution within a single session or context window. They lack explicit mechanisms to transform execution learnings into permanent, cross-project functional libraries, leading to recurring engineering overhead across projects.

B. Tool-Augmented LLMs and Function Calling

Augmenting LLMs with external tools enables models to execute actions beyond text synthesis. Toolformer [4] demonstrated self-supervised learning for tool usage, while ToolLLM [5] scaled tool interaction to thousands of real-world APIs. Despite these advances, tool-use approaches treat external tools as execution extensions of the LLM. In contrast, COSE treats the LLM strictly as a bounded cognitive component embedded within a deterministic control structure, enforcing that deterministic logic remains completely independent of model behavior.

C. Software Engineering Paradigms and Neurosymbolic Integration

Component reusability and functional decoupling are classic principles in software architecture. Microservices [6] promote service-level independence, while the Unix philosophy [7] advocates minimal, composable programs. In the context of AI-driven systems, Kautz [8] emphasized the necessity of integrating neural learning with symbolic reasoning—a principle known as the “third wave” of AI. COSE operationalizes this integration by formalizing a strict separation between deterministic capabilities (symbolic/invariant) and cognitive capabilities (neural/LLM), mediated via persistent, file-based artifacts.

III. THEORETICAL FRAMEWORK & NOTATION

To eliminate ambiguity and establish formal rigor, this section defines the mathematical notation, mathematical relations, and core domain terminology of the COSE methodology.

A. Formal Definitions

- **Problem (P):** A concrete functional requirement whose resolution produces operational value, defined independently of implementation technology.
- **Micro-problem (M):** The minimal, single-responsibility functional unit derived from the decomposition of a problem.
- **Capability (C):** An autonomous, reusable, and deterministic or cognitive functional unit designed to solve a specific micro-problem.
- **Capability Library (L):** The cumulative, non-decreasing set of all validated capabilities available to the organization.
- **Agent (A):** An executable mechanism or runtime entity that instantiates one or more capabilities within an execution pipeline.
- **Pipeline (Π):** An ordered sequence of capabilities executing data transformations, where output artifacts from step i serve as input artifacts for step $i + 1$.
- **Artifact (T):** A persistent, file-based data structure (e.g., JSON schema, CSV tensor, report, image) generated during pipeline execution that guarantees state auditability.
- **Orchestrator (O):** The component responsible for routing data flows, managing state transitions, and sequencing capability execution without embedding domain business logic.

B. Mathematical Notation and Algebra

Let $P \in \mathcal{P}$ represent a target domain problem. The decomposition operator $\text{decompose}(\cdot)$ maps P to an ordered set of micro-problems:

$$\text{decompose}(P) \longrightarrow \{M_1, M_2, \dots, M_n\} \quad (1)$$

For each micro-problem M_i , the resolution function $\text{resolve}(M_i)$ queries the current Capability Library L :

$$\text{resolve}(M_i) = \begin{cases} C_k \in L, & \text{if a matching capability exists} \\ C_{\text{new}}, & \text{otherwise} \end{cases} \quad (2)$$

When a matching capability does not exist in L , a new capability C_{new} is developed and permanently ingested into L via the library expansion operator:

$$L_{t+1} = L_t \cup \{C_{\text{new}}\} \quad (3)$$

Thus, the cardinality of the Capability Library grows monotonically over time:

$$|L_{t+1}| \geq |L_t| \quad (4)$$

A complete software application or workflow is constructed by composing capabilities into an executable pipeline Π :

$$\Pi = \text{compose}(C_1, C_2, \dots, C_n) \quad (5)$$

where data flow between capability nodes is mediated exclusively through persistent artifacts T :

$$\Pi : T_{\text{in}} \xrightarrow{C_1} T_1 \xrightarrow{C_2} T_2 \xrightarrow{\dots} T_{n-1} \xrightarrow{C_n} T_{\text{out}} \quad (6)$$

An agent A_j executes a set of capabilities:

$$A_j \longrightarrow \{C_a, C_b, \dots\} \quad (7)$$

COSE enforces a design preference toward single-capability execution agents (1 Agent $\rightarrow$ 1 Capability) whenever structurally feasible.

Model independence is mathematically defined by stating that replacing the underlying LLM provider ($\text{LLM}_1 \rightarrow \text{LLM}_2$) leaves the methodology and deterministic capability interfaces invariant:

$$\text{replace}(\text{LLM}_1, \text{LLM}_2) \implies \text{Methodology and Pipeline Invariant} \quad (8)$$

IV. ARCHITECTURAL PARADIGM

The COSE architectural framework enforces strict segregation of concerns across system design, runtime execution, and persistence layers.

A. Core Design Principles

- 1) **Single Responsibility and Atomic Granularity:** Each capability C_i solves exactly one micro-problem M_i . Complex behavior must emerge from composition, not from internal capability bloating.
- 2) **Strict Deterministic vs. Cognitive Segregation:**
 - *Deterministic Capabilities:* Executed via immutable code, mathematical algorithms, database queries, or logical rules. Inputs and outputs are predictable and invariant given identical state.
 - *Cognitive Capabilities:* Executed by LLMs for tasks requiring non-deterministic reasoning, ambiguity resolution, or natural language generation.
 - *Segregation Rule:* Any computation that can be executed deterministically must be removed from cognitive units.
- 3) **Artifact-Based State Persistence:** Inter-capability communication occurs strictly through persistent file artifacts stored in the workspace repository. In-memory volatile contexts across execution boundaries are prohibited, ensuring full execution replayability and auditability.
- 4) **LLM as a Bounded Component:** LLMs are treated strictly as isolated cognitive workers invoked at specific pipeline nodes, rather than acting as unconstrained, autonomous system controllers.
- 5) **Model Independence and Interchangeability:** Cognitive capabilities interface through standardized client abstractions, enabling runtime swapping between local open-weight models (e.g., Ollama/DeepSeek) and commercial cloud APIs (e.g., OpenAI/Gemini) without modifying surrounding capabilities or orchestration logic.

B. The COSE Execution Lifecycle

Fig. 1 illustrates the recursive operational cycle of COSE.

V. EMPIRICAL EVALUATION & CASE STUDIES

To validate the COSE methodology against real-world complexity, we evaluate its implementation across seven production systems developed within our engineering infrastructure.

A. Case Study 1 — Mammon: Algorithmic Trading Platform

1) *Problem Formulation:* Build an automated quantitative trading platform capable of data acquisition, candle validation, technical signal generation, paper execution, and historical backtesting, while isolating trading strategies as interchangeable components from underlying execution infrastructure.

2) *Decomposition & Pipeline Construction:* The system was decomposed into modular capabilities communicating via disk artifacts:

- `market.price_history`: Binance Spot REST API integration for raw OHLCV fetch.
- `market.candle_repository`: Deterministic candle validation, integrity check, and unified serving.
- `signal.entry`: Technical indicator strategy (RSI, MACD, Moving Averages) returning BUY/SELL/NEUTRAL.

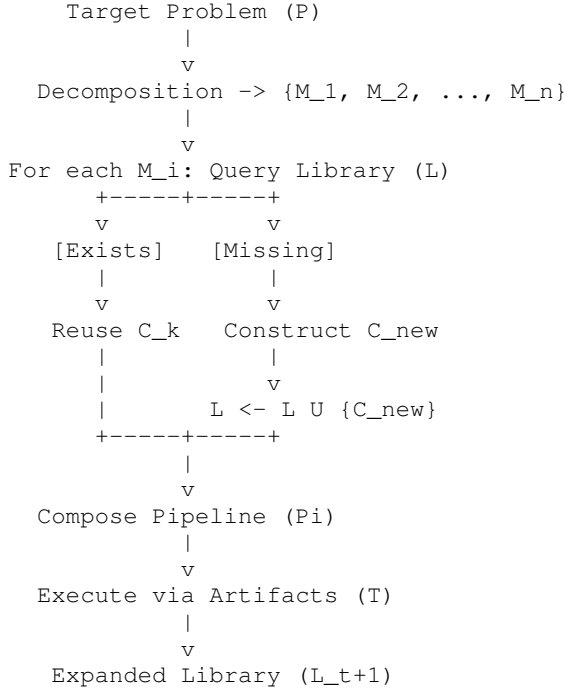


Fig. 1. The COSE System Lifecycle: Problem decomposition, capability retrieval/ingestion, pipeline composition, and artifact-based resolution.

- `execution.paper`: Simulated order matching, portfolio cash accounting, and order state persistence.
- `simulation.runner`: Historical backtesting engine over the complete pipeline.

Binance REST → price_history → candle_repo → Indicators → `signal_mapper.execution.paper` (9)

Mammon formally segregated operational data pipelines from knowledge-generation pipelines:

- **Operational Pipeline**: Fetch → Validate → Analyze → Execute.
- **Knowledge Pipeline**: Audit → Repository → Statistics → Interpretation → Hypotheses.

3) *Empirical Validation (36-Configuration Stress Matrix)*: To evaluate architectural stability, Mammon was subjected to a comprehensive stress matrix encompassing **36 distinct experimental configurations**:

- **Assets (4)**: BTCUSDT, ETHUSDT, SOLUSDT, BNBUSDT.
- **Timeframes (3)**: 15m, 1h, 4h.
- **Sample Windows (3)**: 200, 500, 1000 candles.

Results: The capability infrastructure achieved a 100% completion rate across all 36 configurations with zero infrastructure or state-desynchronization failures. Furthermore, during live paper execution on ETHUSDT (1h timeframe), a live data ingestion bug (frozen candle snapshot) was detected and repaired on August 10, 2026, by refactoring `simulation.runner` to pull fresh REST candles per cycle

TABLE I
MAMMON INFRASTRUCTURE VALIDATION STRESS MATRIX (36 CONFIGURATIONS)

Asset	Timeframe	Window (Candles)	Exec. Status	Infra. Failures
BTCUSDT	15m, 1h, 4h	200, 500, 1000	100% Pass	0
ETHUSDT	15m, 1h, 4h	200, 500, 1000	100% Pass	0
SOLUSDT	15m, 1h, 4h	200, 500, 1000	100% Pass	0
BNBUSDT	15m, 1h, 4h	200, 500, 1000	100% Pass	0

without modifying downstream signal or execution capabilities. The strategy’s selectivity principle—where returning NEUTRAL is treated as a valid decision—was empirically confirmed.

B. Case Study 2 — Knowledge Factory: Cognitive Distillation Pipeline

1) *Problem Formulation*: Transform raw development activity logs into structured organizational knowledge cards using a hybrid deterministic-cognitive pipeline.

2) *Architecture & JSON Tensors*: Knowledge Factory implements an 11-stage pipeline separating deterministic evidence collection from LLM cognitive synthesis:

Loops → LoopParser → LoopTensor → EvidenceExtractor → EvidenceBuilder → ProjectTensor → PromptBuilder → LLMClient → KnowledgeParser → KnowledgeCard (10)

All intermediate data exchange is enforced via strict JSON Schemas representing structured tensors: LoopTensor, EvidenceTensor, ProjectTensor, and KnowledgeCard. Routing is dynamically resolved via `signal_mapper.execution.paper`

3) *Cognitive Interchangeability*: The LLM interaction layer implements a polymorphic client interface (BaseClient, OllamaClient, OpenAIClient, LLMRouter, LLMClient). In empirical runs, switching between local Ollama models and cloud APIs produced valid KnowledgeCard artifacts with zero modifications to deterministic upstream parsers, demonstrating strict model independence.

C. Case Study 3 — Website Intelligence Factory (WIF): Web Reverse Engineering

1) *Problem Formulation*: Automatically discover the technical architecture of arbitrary target websites, select optimal extraction strategies, generate declarative extraction profiles, and produce automated web crawlers.

2) *Pipeline Implementation*: WIF executes a multi-stage discovery flow:

Target → Capture → Normalize → Inspect → Fusion → Profile → Generate (11)

3) *Deterministic-Cognitive Boundary*: All network capture, DOM parsing, header analysis, robots.txt inspection, and crawler template rendering operate 100% deterministically.

LLM calls are strictly restricted to resolving structural ambiguity during extraction profile selection. The resulting deliverable is not a fragile scraper script, but a highly reusable *Extraction Profile* artifact.

D. Case Study 4 — AI Middleware: Multi-Model Orchestration Bridge

1) *Problem Formulation*: Coordinate heterogeneous cognitive models (e.g., ChatGPT, Gemini, DeepSeek) to interact safely with file-system capabilities and local project artifacts.

2) *Implementation*: The Middleware acts as an orchestrator exposing deterministic file capabilities (`read_file`, `write_file`, `list_files`, `diff_files`, `get_snapshot`) mediated by MapRouter. Cognitive models operate as interchangeable reasoning workers, issuing function calls while execution and state persistence remain strictly deterministic.

E. Case Studies 5, 6, & 7 — Business Domain Applications

- **Dol-Goldur (Catalog Automation)**: Automated enrichment, resource retrieval, curation, and publishing of music catalogs. Decomposed into 6 independent capabilities, expanding the general capability library.
- **CatalogVision (Computer Vision Inventorying)**: Image-to-inventory processing pipeline using computer vision models, data normalization, and aggregation capabilities.
- **Andigraf (Competitive Intelligence)**: Automated market intelligence platform performing catalog normalization, product matching, price trend crawling, and report generation.

Cases 5–7 confirm that capabilities developed for technical/financial domains (e.g., normalization, crawling, report generation) are directly reusable in commercial business workflows without architectural modification.

VI. COMPARATIVE ANALYSIS & DISCUSSION

A. Comparison with Existing Paradigms

Table II presents a systematic comparison between COSE and established software engineering and agentic paradigms.

B. Discussion of Empirical Findings

The empirical results across Mammon, Knowledge Factory, WIF, Middleware, and business applications validate the core research hypothesis: *treating atomic capabilities as the primary unit of development enables systematic reuse and reduces marginal engineering cost*.

Key empirical insights include:

- 1) **Infrastructure Stability under Stress**: The 36-configuration stress test in Mammon proved that decoupling execution infrastructure from strategy logic isolates algorithmic risk to the decision node without exposing system stability to runtime failures.
- 2) **Cost and Latency Optimization via Segregation**: By restricting LLM calls strictly to ambiguous or cognitive nodes in WIF and Knowledge Factory, the majority

of pipeline transformations are executed in the deterministic domain, with the LLM invoked only where reasoning, judgment, or ambiguity resolution is required. Quantitative measurement of the deterministic/cognitive ratio is deferred to the metrics defined in Section VII and the recommended metrics in the appendix.

- 3) **Auditability via Persistent Artifacts**: Disk-based JSON tensors ensured complete execution replayability, enabling offline debugging and cross-model comparison without volatile context drift.

C. Limitations

While COSE demonstrates strong empirical benefits, several limitations warrant consideration:

- **Artifact Storage Overhead**: High-frequency artifact persistence (e.g., tick-level market data) introduces disk I/O overhead requiring specialized storage buffering.
- **Library Governance**: As $|L| \rightarrow \infty$, automated capability discovery and garbage collection mechanisms become necessary to prevent capability duplication.
- **Quantitative Marginal-Cost Measurement**: The reduction of marginal engineering cost is argued qualitatively from the case studies; a formal quantitative model of marginal-cost decay remains future work (see Section VII).

VII. CONCLUSION

This paper presented *Capability-Oriented Software Engineering* (COSE), an incremental software engineering methodology that shifts the primary organizational product from transient software applications to a permanent, evolving library of reusable capabilities. Through formal mathematical definitions, deterministic-cognitive decoupling, artifact-based persistence, and dynamic pipeline composition, COSE guarantees architectural stability and model independence.

Empirical evaluation across seven production systems—including algorithmic trading (Mammon 36-configuration stress test), knowledge distillation, web reverse engineering, multi-model middleware, and commercial vision applications—demonstrates that COSE effectively decouples infrastructure from domain logic, enforces deterministic bounds on LLMs, and progressively lowers the marginal cost of building complex software solutions.

Future research will focus on formal mathematical models for marginal cost decay, automated semantic capability retrieval, automated garbage-collection protocols for enterprise capability libraries, and quantitative metrics for the deterministic/cognitive execution ratio across capabilities and pipelines.

REFERENCES

- [1] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Chichester, UK: John Wiley & Sons, 2009.
- [2] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023, pp. 1–33.
- [3] Q. Wu, G. Bansal, S. Zhang, Y. Wu, B. Li, L. Tan, and C. Wang, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.

TABLE II
SYSTEMATIC PARADIGM COMPARISON: COSE VS. ESTABLISHED ARCHITECTURES

Architectural Dimension	Classic MAS [1]	Modern Agentic [2], [3]	Microservices [6]	Unix Philosophy [7]	COSE Methodology
Primary Entity	Autonomous Agent	Planner / Worker	Service Boundary	OS Executable	Atomic Capability (C)
State Storage	Volatile / Message	Context Window	Database	Standard I/O / Pipes	Persistent File Artifact (T)
LLM Coupling	High / Core Engine	High / Core Engine	N/A	N/A	Decoupled / Interchangeable
Reusability Target	Agent Behaviors	Workflows / Prompts	Business Services	System Utilities	Global Capability Library (L)
Execution Structure	Social Interaction	Orchestrated Graph	REST / gRPC	CLI Pipelines	Artifact-Driven Pipelines (II)
Cumulative Growth Rule	Implicit	Optional	Explicit (Services)	File System	Mandatory ($L_{t+1} \geq L_t$)

- [4] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Proc. Advances in Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 68539–68551.
- [5] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, “ToolLLM: Facilitating large language models to master 16,000+ real-world APIs,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024, pp. 1–24.
- [6] M. Fowler and J. Lewis, “Microservices: A definition of this new architectural term,” *ThoughtWorks*, Mar. 2014. [Online]. Available: <https://martinfowler.com/articles/microservices.html>](<https://martinfowler.com/articles/microservices.html>)
- [7] E. S. Raymond, *The Art of Unix Programming*. Boston, MA, USA: Addison-Wesley Professional, 2003.
- [8] H. Kautz, “The third wave of AI: Integration of neural and symbolic reasoning,” *AI Magazine*, vol. 43, no. 1, pp. 105–112, 2022. doi: 10.1002/aaai.12040.